



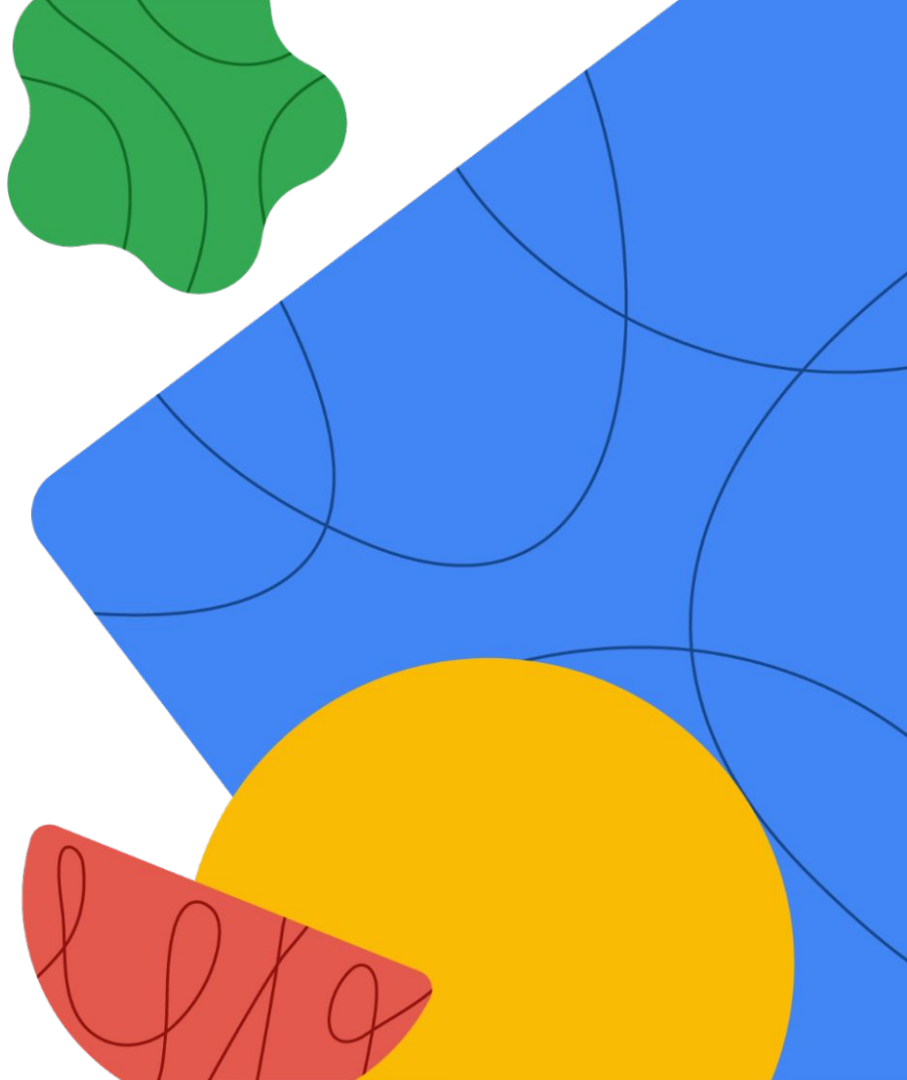
UCLA CS GRADUATE SEMINAR • FALL 2026

Scaling LLMs on Cloud TPUs

Architectures, Distributed Systems,
and the MaxText Ecosystem

Hengtao Guo

Google Cloud TPU & MaxText Team



About Me & What I Do at Google



OSS Model Bring-Up

- **SOTA Model Porting:**

Qwen 3/3.5, Llama 4, Gemma 3/4

- **Numerical Parity:**

Layer-wise assert_allclose verification

- **Checkpoint Conversion:**

Bi-directional Safetensors ↔ Orbax

- **Agentic Automation:**

Automated porting via AI agent skills



Multimodal & Training

- **Multimodal VLM / Omni:**

Vision & audio encoders in MaxText

- **Projection Layers:**

MLP & cross-attention connectors

- **Post-Training:**

Instruction tuning & preference alignment

- **Reasoning RL (GRPO):**

Verifiable rewards for coding & math



High-Perf Systems

- **MaxText Core Engine:**

Pure JAX/Flax high-efficiency runtime

- **4D Parallelism:**

FSDP, Tensor, Pipeline, & Context sharding

- **Sparse MoE Scaling:**

Expert Parallelism (EP) across pods

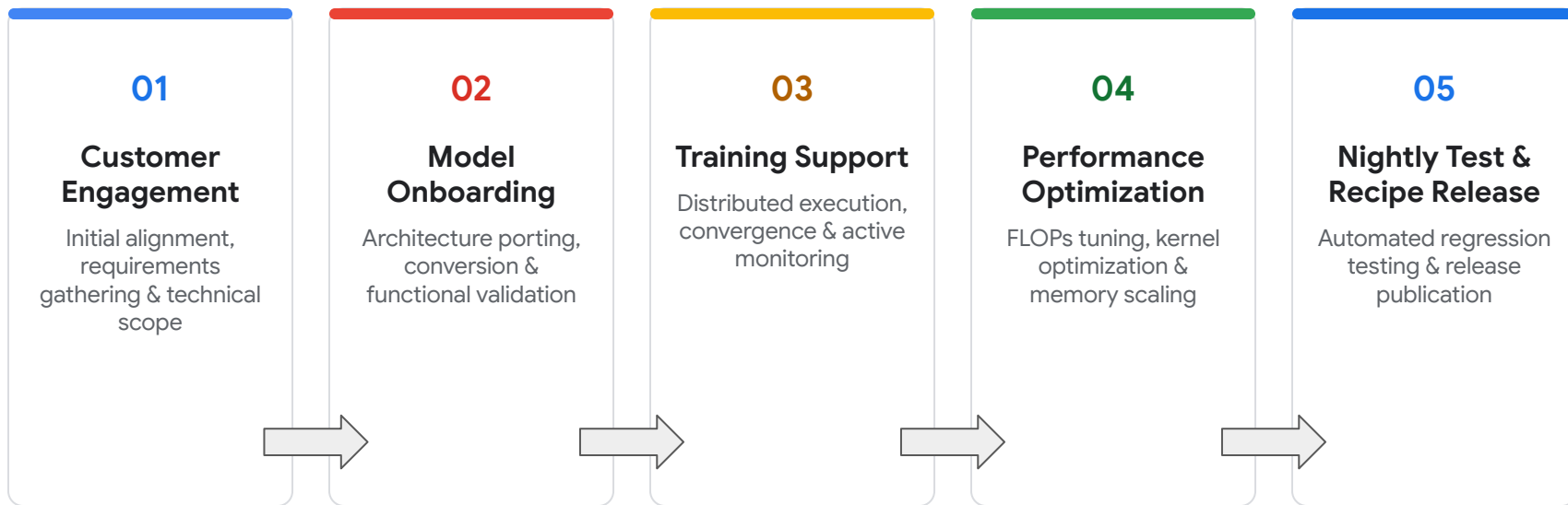
- **TPU Kernel Tuning:**

Pallas FlashAttention & >60% MFU



Focus: Bridging cutting-edge open models with high-throughput distributed TPU systems

MaxText Team Typical Workflow



COURSE ROADMAP

Agenda

From TPU silicon & XLA compiler foundations to distributed scale and coding intelligence.

01

Introduction to Cloud TPUs & MaxText

What is MaxText, why TPUs matter

02

MaxText Model Zoo & Architectures

Dense vs MoE, attention variants, multimodal & bring-up

03

Distributed Scale & 4D Parallelism

"Too big to fit", FSDP, Tensor & Expert Parallelism, hybrid sharding mesh

04

Training Lifecycles: Pre- & Post-Training

Pretrain loss/MTP, SFT completion-only, verifiable RL: GRPO

05

Hands-On Demo

Cloud TPU Colab



MaxText & TPU

What is **MaxText**: High-MFU LLM Training Tutorial on TPUs

Core Design Philosophy

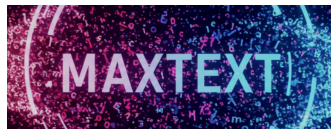
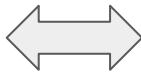
- **Pure JAX & Flax:** Simple, transparent Python with zero heavy wrapper layers.
- **North-Star Metric (MFU):** Consistently achieves 55%–65%+ Model FLOPs Utilization.
- **Zero Bloat:** Easy to inspect tensor shapes, sharding annotations, and memory.
- **Effortless Scaling:** Same code runs on 1 TPU VM (8 chips) or 4,096+ chip pods.

Production & OSS Capabilities

- **Open Source:** Apache 2.0 licensed at github.com/google/maxtext.
- **End-to-End Lifecycle:** Pretraining, SFT, RL (GRPO), and low-latency decode.
- **Checkpoint Interop:** Bi-directional conversion with HuggingFace Safetensors.
- **Automated Nightly CI:** Continuous regression benchmarks on v5p, v6e, and v7x.



Hugging Face



Goal: A high-performance, transparent, and hackable reference LLM engine in pure JAX

Hardware-Software Co-Design: The TPU Stack

Layer 4: MaxText (Model Engine)

- **Pure JAX/Flax LLM & VLM engine** • >60% MFU at scale • Out-of-the-box 4D parallelism & SFT/RL recipes

Layer 3: JAX & Flax (Framework)

- **Functional transformations:** jit, grad, vmap • Declarative SPMD sharding (Mesh & NamedSharding)

Layer 2: XLA (Compiler)

- **Whole-graph compilation** • Deep operator fusion • Automatic HBM-to-VMem memory scheduling

Layer 1: TPU Silicon (Hardware)

- **2D Systolic MXUs** • Vector Processing Units (VPUs) • High Bandwidth Memory • Optical ICI

Tight co-design: Software abstractions map 1:1 to compiler primitives and hardware execution units

The Scale of Compute: Why TPUs Matter



Gemini
Text generation

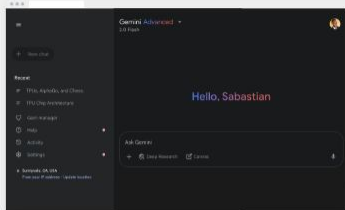
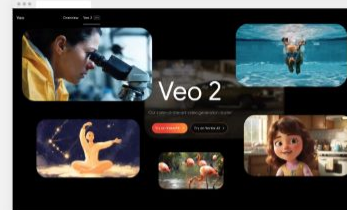


Imagen
Image generation



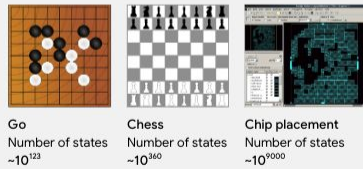
Veo
Video generation



AlphaFold
Protein 3D structure prediction



AlphaGo/Zero
Human-rivaling gameplay



AlphaChip
Chip-designing agent



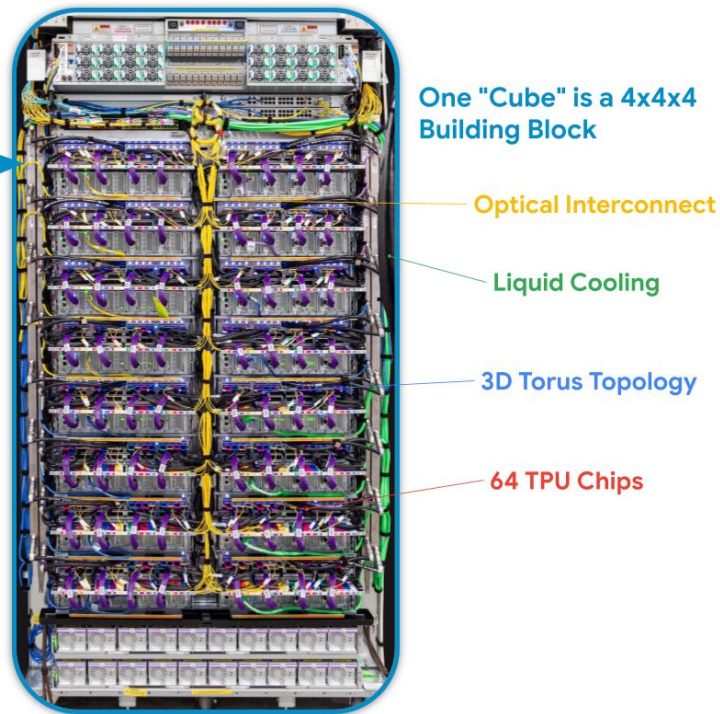
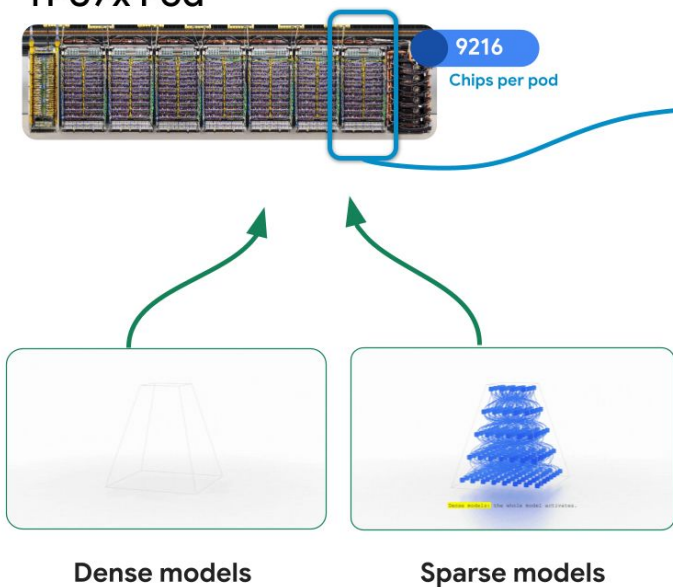
TPU Generations

Specification	TPU7x	TPU v6e	TPU v5p
Number of chips per pod	9216	256	8960
Peak compute per chip (BF16) (TFLOPs)	2307	918	459
Peak compute per chip (FP8) (TFLOPs)	4614	918	459
HBM capacity per chip (GiB)	192	32	95
HBM bandwidth per chip (GBps)	7380	1638	2765
Number of vCPUs (4-chip VM)	224	180	208
RAM (GiB) (4-chip VM)	960	720	448
Number of TensorCores per chip	2	1	2
Number of SparseCores per chip	4	2	4
Bidirectional inter-chip interconnect (ICI) bandwidth per chip (GBps)	1200	800	1200
Data center network (DCN) bandwidth per chip (Gbps)	100	100	50

TPU 3D Torus Interconnect & Optical Circuit Switching (OCS)

Interchip Interconnect (ICI) In Action

TPU7x Pod

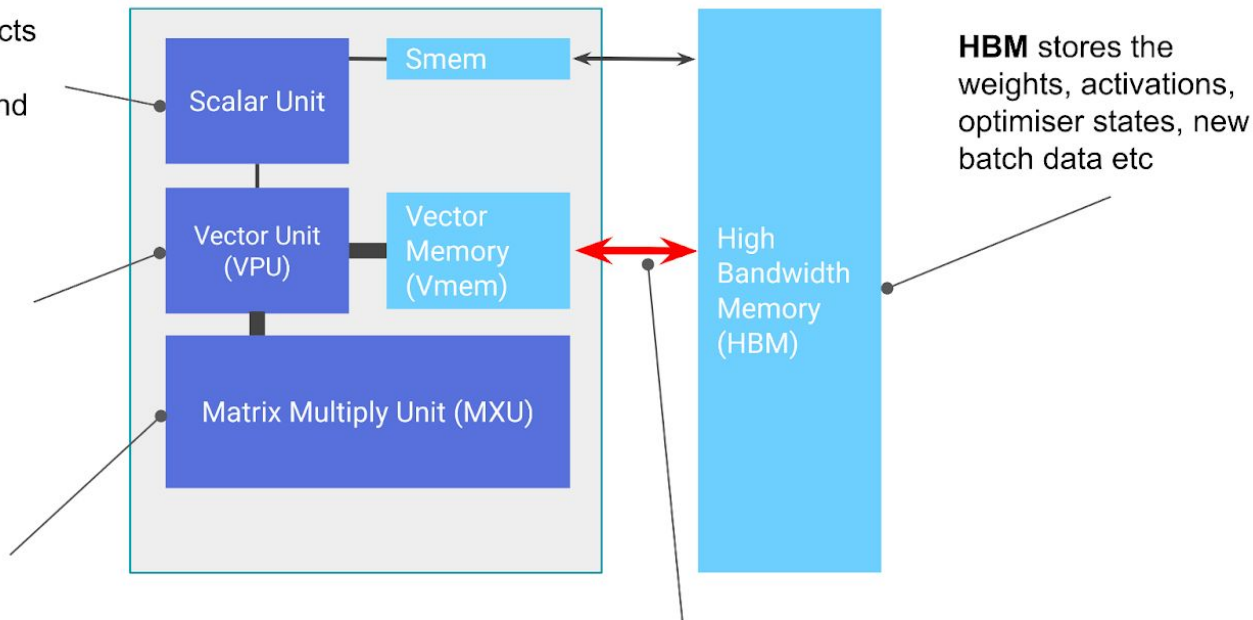


TPU Compositions

The **Scalar Unit** sort of acts like a CPU 'dispatching' instructions to the VPU and MXU

The **VPU** performs elementwise operations (e.g. activations), loads data into the MXU

The **MXU** performs matrix multiplications - and is therefore our driver of chip FLOP/s.



Abstract layout of a TPU TensorCore.

HBM stores the weights, activations, optimiser states, new batch data etc

HBM bandwidth: determines how fast data goes to and from the computational elements

TPU Compositions

Scalar Unit

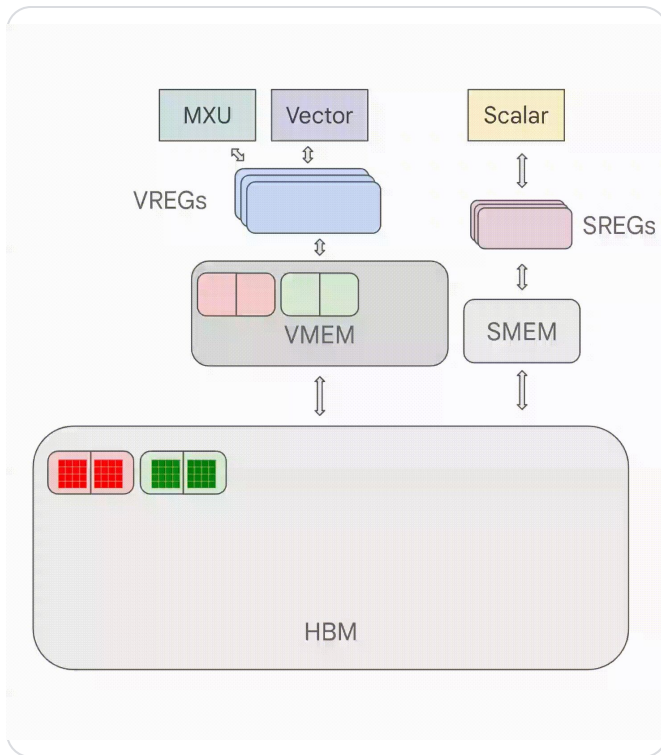
Acts like a CPU, dispatching control instructions to Vector and MXU modules.

Vector Unit (VPU)

Executes elementwise calculations, activation functions, and streams data to MXU.

Matrix Unit (MXU)

Drives core compute by executing massive parallel matrix multiplication FLOPs.



Registers (VREGs / SREGs)

Ultra-low latency storage backing vector and scalar computation pipelines.

On-Chip Memory (VMEM/SMEM)

High-speed local SRAM scratchpads for fast operand staging.

High Bandwidth Memory (HBM)

Main memory holding model weights, activations, optimizer states & batch data.

A decorative graphic on the left side of the slide. It features a large blue circle with a white number '02' in the center. Above the blue circle is a green circle with a yellow star-like shape overlapping it. Below the blue circle are a yellow circle and a red circle, both partially visible.

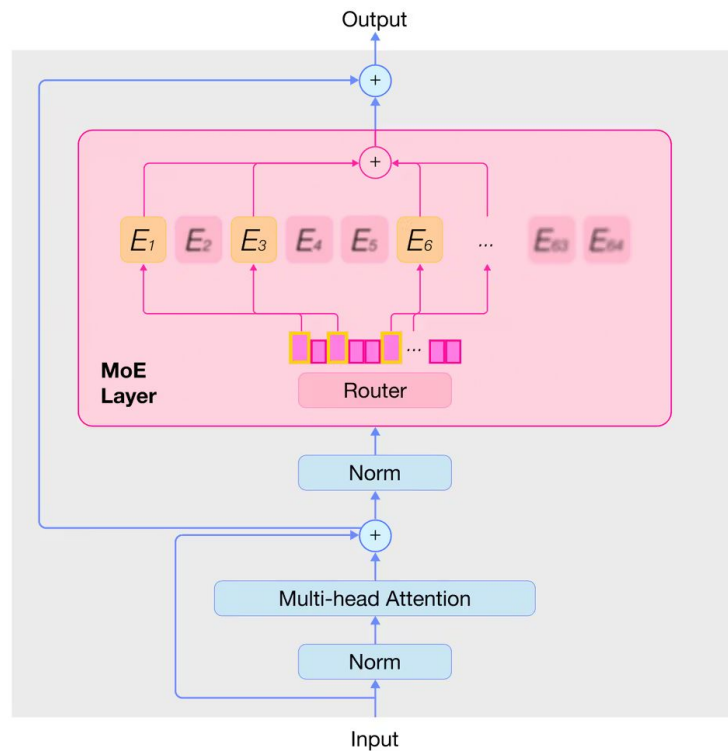
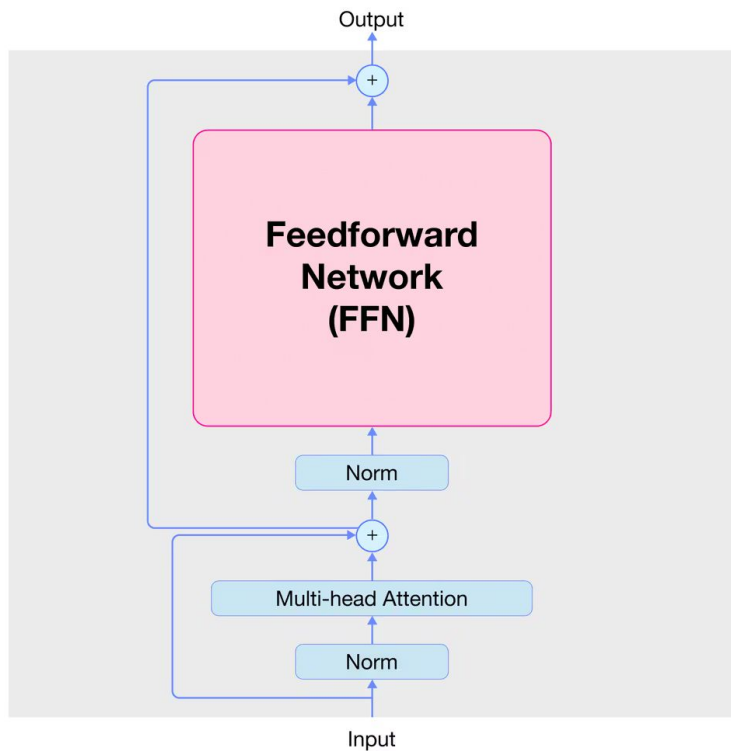
02

Model Zoo & Bringups

MaxText Model Zoo: Supported Architectures

MODEL FAMILY	VARIANTS & SCALES	ATTENTION MECHANISM	KEY FEATURES & OPTIMIZATIONS
Gemma (Google) Dense, MoE & VLM	Gemma 1 (2B/7B), 2 (2B-27B), 3 (4B-27B), 4 (26B MoE, 31B Dense)	GQA, Local-Global SWA, Bidirectional Image Mask	QK-Norm, Value Norm, routed+shared experts, SigLIP ViT
Llama (Meta) Dense, MoE & VLM	Llama 2 (7B-70B), 3/3.1/3.3 (8B-405B), 4 (Scout 109B, Maverick 400B)	Grouped-Query Attention (GQA), RoPE	SwiGLU, QK-Norm, routed experts (Llama 4), multimodal projector
DeepSeek (AI) Sparse MoE	V2 (16B/236B), V3/3.1/3.2 (671B), R1 (671B), V4 Flash (284B)	Multi-Head Latent Attention (MLA), Sparse Attention	Multi-Token Prediction (MTP), Qwix FP8 quantization, HashRouting
Qwen (Alibaba) Dense, MoE & VLM	Qwen 2.5 (1.5B-72B), 3 Dense (0.6B-32B), 3 MoE (30B-480B), 3.5 (397B)	GQA, GatedDeltaNet, Hybrid Attention	Tokamax GMM MoE kernel, QK-Norm, SwiGLU, Grain VLM pipeline
Kimi & GPT-OSS Sparse MoE (Extreme)	Kimi K2 / K2-Thinking (1T), K2.5/2.6; GPT-OSS (20B, 120B)	DeepSeek V3 style MLA, Attention Sinks	MuonClip optimizer, long-context scaling, RL post-training

Dense v.s. Mixture-of-Experts



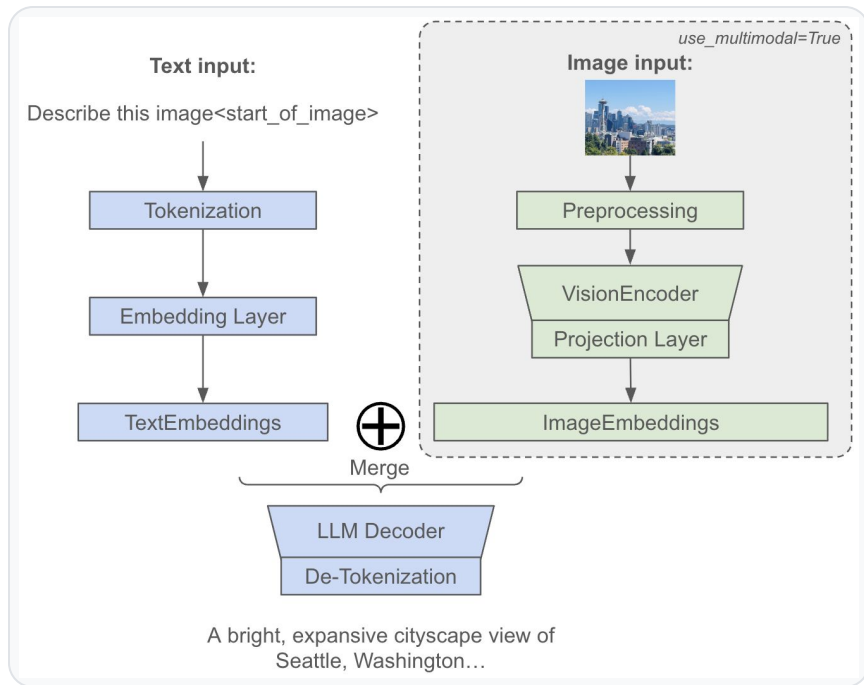
MaxText Multimodal Architecture

VISUAL PROCESSING & ENCODING

- **Perceive Beyond Text:** Incorporate image, video, audio, and more.
- **Vision Encoder:** Splits images into patches and generates highly representative embeddings.
- **Projector (Trainable):** Maps visual embeddings directly into the core text embedding space.

CONFIGURATION & ACTIVATION

- **Activation:** `use_multimodal=True` in config.
- **Gradient Control:** `freeze_vision_encoder_params=True` (default mechanism).



The 4-Phase Model Bring-Up Methodology

Phase 1: Mini-Slice

- **Audit HF Source:** Identify custom attention or norm ops.
- **2-Layer Slice:** Save tiny checkpoint in /dev/shm.
- **Fast Iteration:** 2-second test cycles without 100GB+ weights.

Phase 2: Layer Parity

- **Build Flax Layers:** Match reference math.
- **Direct Weight Copy:** Load PyTorch tensors into JAX.
- **Strict Assertion:**
`assert_allclose(1e-5)`

Phase 3: Mappers

- **Central Mapping:** Register hooks in param_mapping.py.
- **Shape Transforms:** Transpose linear weights $[O,I] \rightarrow [I,O]$.
- **Scan Stacking:** Stack layers for nn.scan.

Phase 4: E2E Check

- **Logits Parity:** Full forward pass $KL \leq 1e-3$.
- **Greedy Decode:** Verify exact token ID match over 16 steps.
- **Full Scale:** Run full checkpoint on TPU pod.

Zero-Regression Guarantee: Every ported model mathematically replicates reference behavior

Common Pitfalls in Model Porting & Solutions

Trap 1: Weight Transposition

- **Layout Mismatch:** PyTorch `nn.Linear` is [out, in]; JAX `DenseGeneral` is [in, out].
- **Silent Bug:** Square matrices (in == out) run without shape errors but output noise!
- **Fix:** Always transpose (.T) linear weights in conversion hooks.

Trap 2: RoPE Permutations

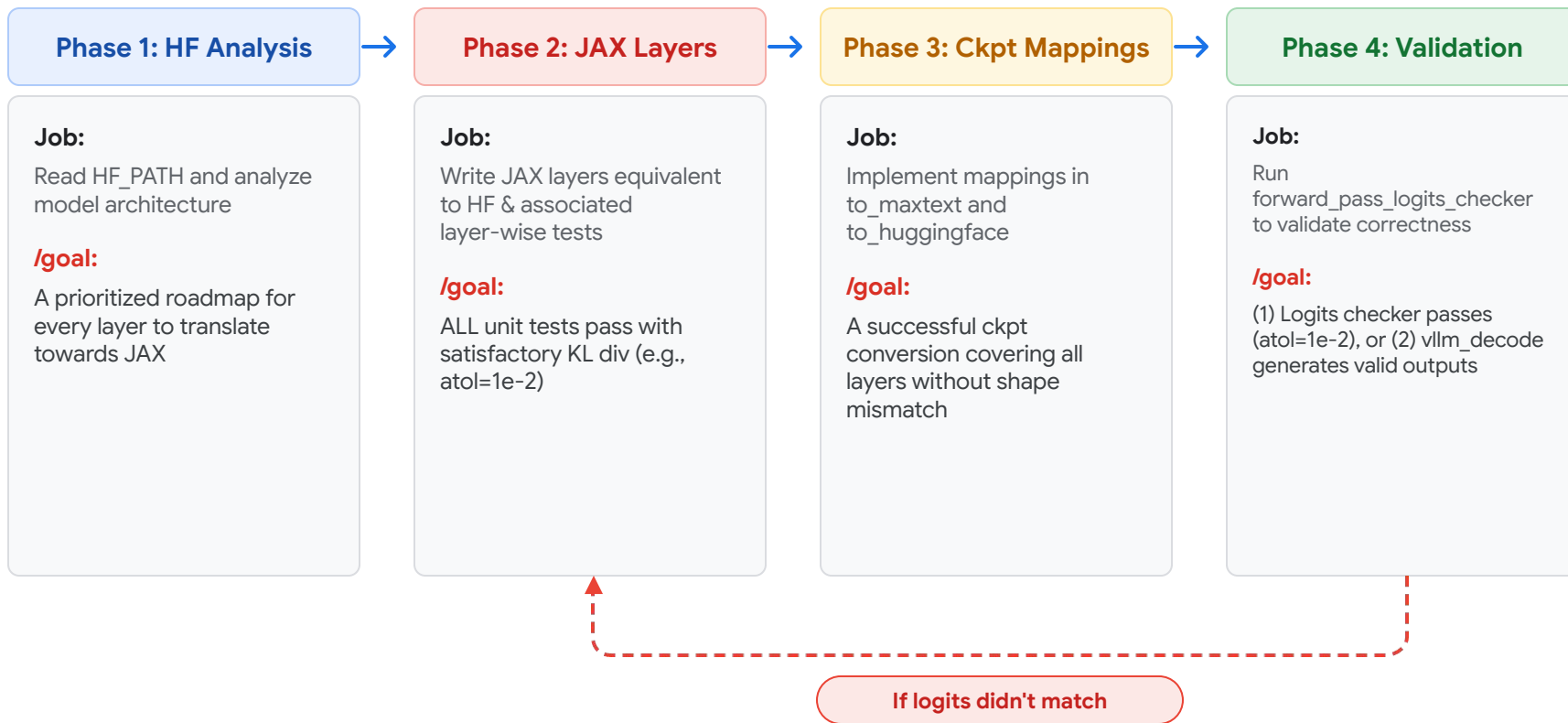
- **Split vs Interleaved:** HF splits head dim [d/2, d/2]; MaxText rotates adjacent pairs.
- **Symptom:** Attention scores drift on long contexts.
- **Fix:** Explicitly permute Q/K projection weights or RoPE sinusoids.

Trap 3: Scanned Layers (nn.scan)

- **10x Faster Compile:** MaxText uses `nn.scan` over transformer layers.
- **Weight Stacking:** Stack per-layer weights into [num_layers, ...].
- **Hybrid Architectures:** Use modulo indexing for alternating dense/MoE or local/global layers.

Takeaway: Layer-wise unit testing isolates subtle numerical bugs before full-model runs

@model_bringup (Entry point • One skill to rule them all)



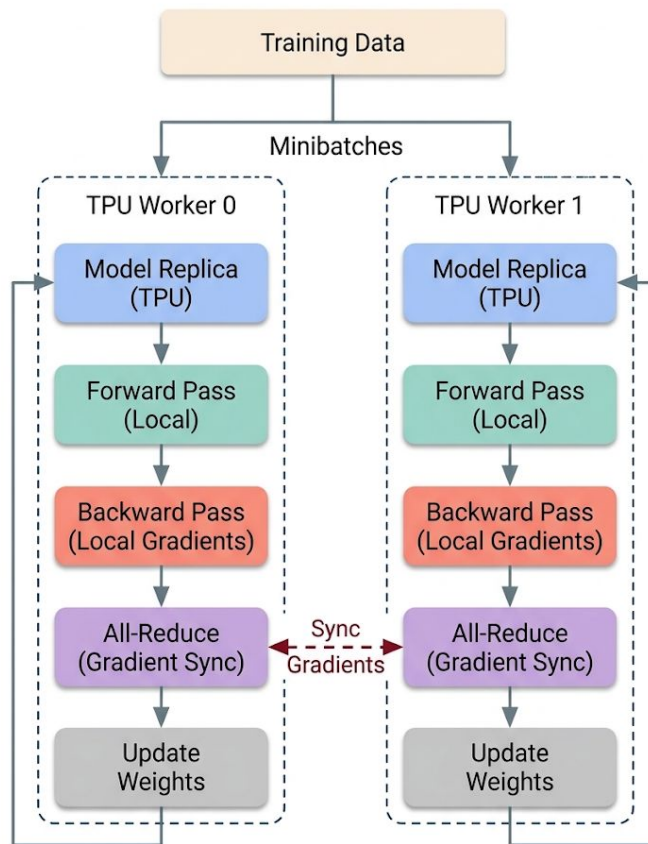


Performance Optimization

Traditional Data Parallelism

- **Full Model Replication:** Every worker hosts an identical, complete copy of model weights, optimizer states, and gradients.
- **Minibatch Partitioning:** The global training dataset is split across workers, processing distinct minibatches independently.
- **All-Reduce Synchronization:** Local gradients computed during backward pass are synchronized across TPUs before weight updates.
- **Capacity Bottleneck:** Limited by single-device memory capacity because the full model must fit entirely on each worker.

Data Parallelism in TPU Training



Mental Model for Training an LLM

100B MODEL • BF16 PRECISION

$$\text{Static Memory} = (2 + 2 + 12) \text{ Bytes} \times 100\text{B} = 1,600 \text{ GB}$$

2 Bytes / weight

1. Model Weights

Parameters stored in BF16 format during training execution.

200 GB

2 Bytes / gradient

2. Gradients

Backpropagation gradient memory requirement per parameter.

200 GB

12 Bytes / state

3. Adam Optimizer

- 4B master weights (FP32)
- 4B momentum (FP32)
- 4B variance (FP32)

1,200 GB

DYNAMIC ACTIVATIONS

Leave extra 50% overhead for data batches, KV cache, and intermediate activations.

Total Memory Required

$$1,600 \text{ GB} \times 1.5 = 2.4 \text{ TB}$$

Distributed Scale: 4D Parallelism on TPUs, and more

1. Data Parallelism (DP)

- **Standard Replicas:** Full model replica mirrored on each device or rank.
- **Gradient Sync:** AllReduce averages gradients across data replicas per step.

2. Fully Sharded Data Parallelism (FSDP)

- **ZeRO-3 Sharding:** Shards weights, gradients & optimizer states across devices.
- **On-the-Fly AllGather:** Materializes layer weights just-in-time for compute.

3. Tensor Parallelism (TP)

- **Intra-Layer Split:** Megatron-style column-wise & row-wise matrix sharding.
- **High-Speed ICI:** Mapped strictly onto optical Inter-Chip Interconnect.

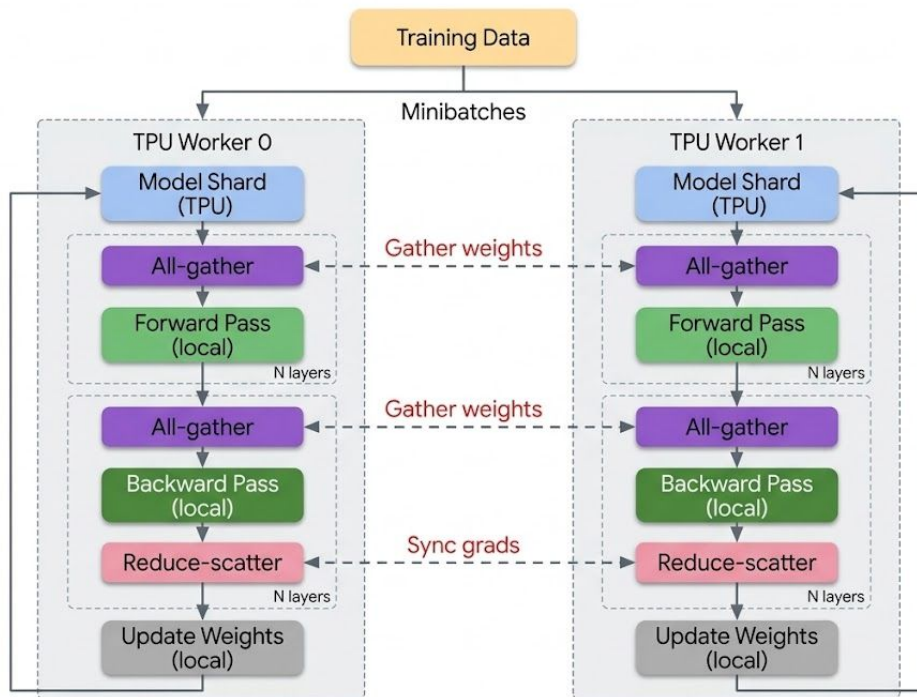
4. Pipeline Parallelism (PP)

- **Inter-Layer Partitioning:** Splits model layers sequentially across pipeline stages.
- **Micro-Batching:** Overlaps execution stages to minimize idle pipeline bubbles.

Fully Sharded Data Parallelism

Fully Sharded Data Parallel (FSDP) Training

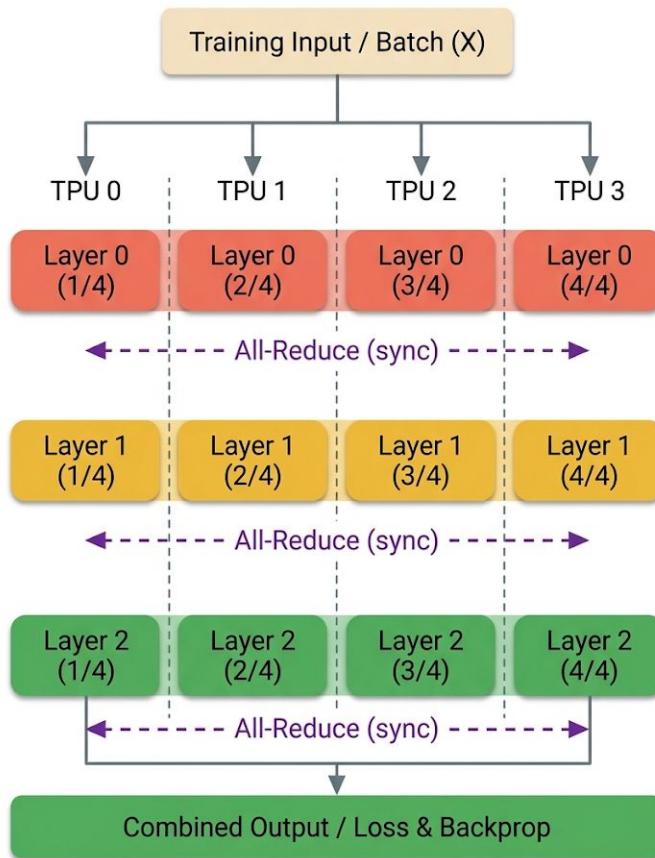
- **Zero-Redundancy Sharding:** Shards parameters, gradients, and optimizer states across devices to eliminate memory redundancy.
- **On-Demand All-Gather:** Gathers full layer weights just-in-time for forward computation, then releases them.
- **Reduce-Scatter Gradients:** Computes and synchronizes gradients so each worker only updates its local shard.
- **Massive Model Scaling:** Reduces per-device memory from $O(\text{Model})$ to $O(\text{Model}/N)$, enabling 100B+ models on limited VRAM.



Tensor Parallelism

- **Intra-Layer Sharding:** Splits individual weight matrices (e.g. attention QKV projections and MLP blocks) across TPUs.
- **Synchronous All-Reduce:** Executes collective All-Reduce operations after each layer to aggregate partial matrix products.
- **High-Speed ICI Required:** Mapped strictly across low-latency, high-bandwidth Inter-Chip Interconnects within host/pod slices.
- **Proportional Memory Relief:** Divides both parameter and activation memory per device for extra-wide layer architectures.

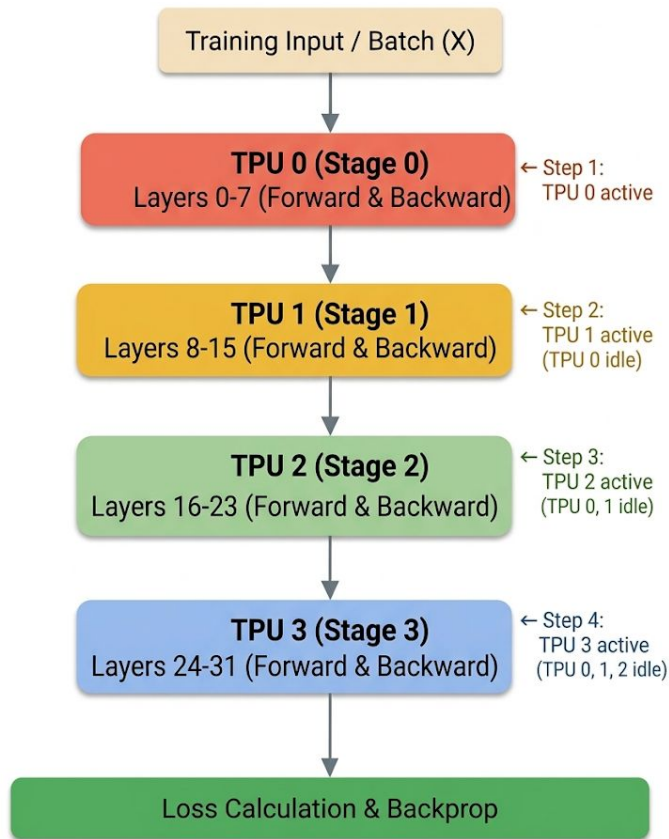
Tensor Parallelism (TP=4) in TPU Training



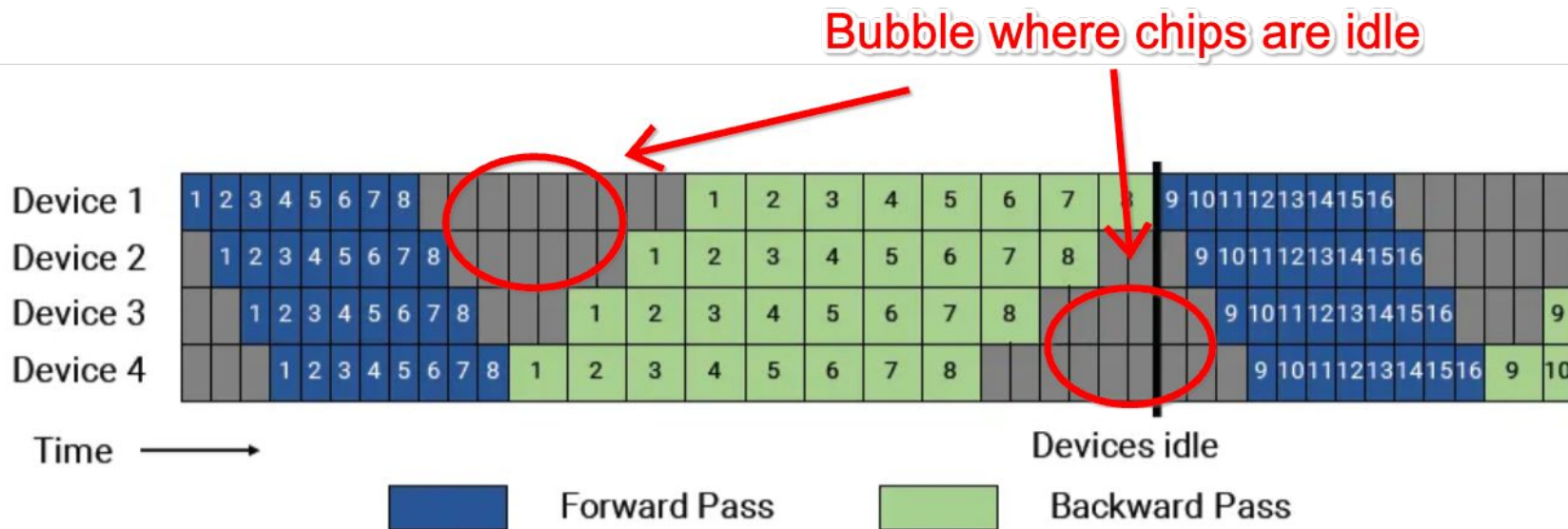
Pipeline Parallelism

- **Inter-Layer Partitioning:** Partitions consecutive model layers into sequential stages assigned to dedicated TPUs (e.g. Layers 0–7 on TPU 0).
- **Point-to-Point Handoff:** Passes intermediate activations forward and backprop gradients backward between neighboring pipeline stages.
- **Pipeline Bubble Overhead:** Sequential dependencies create idle device time during pipeline fill (ramp-up) and drain phases.
- **Micro-Batch Scheduling:** Splits training minibatches into micro-batches (e.g. 1F1B schedule) to overlap stage execution and boost TPU utilization.

Pipeline Parallelism in TPU Training



Pipeline Parallelism



JAX Foundations: The Functional Triad (grad, jit, vmap)

1. Autodiff (jax.grad)

- Reverse-Mode Differentiation:

Computes exact vector-Jacobian products through arbitrary Python logic.

- Explicit State Passing:

```
(new_params, loss) = f(params, x, y)
```

- Code Snippet:

```
grads = jax.grad(loss_fn)(params, x, y)
```

2. Compilation (jax.jit)

- Ahead-of-Time Graph Compilation:

Traces pure functions into intermediate representation (jaxpr) for XLA.

- Eliminates Python Runtime:

Zero interpreter overhead during execution steps.

- Code Snippet:

```
step_fn = jax.jit(train_step)
```

3. Vectorization (jax.vmap)

- Automatic Vectorization:

Transforms single-example math to batches without manual slicing.

- Arbitrary Composition:

```
jit(vmap(grad(loss_fn)))
```

- Code Snippet:

```
batch_pred = jax.vmap(predict, (None, 0))
```

Core Principle: Pure mathematical functions without side effects enable global compiler optimization

Attention Mechanisms

Attention Variants: GQA & MLA

- Grouped-Query Attention (GQA):

Shares KV heads across query groups (e.g., 8:1), slashing KV-cache bandwidth during decode.

- DeepSeek Latent Attention (MLA):

Compresses KV into low-rank latent vector:

$\text{Proj} = [\text{B}, \text{D}] @ [\text{D}, \text{R}] @ [\text{R}, \text{Q}, \text{H}]$

Shrinks KV-cache memory footprint by >85% without quality degradation.

TPU Sparsity & Block Tiling

- Block Sparsity on TPUs:

Optimal 2D tile blocks (e.g. 2k on TPU v5p/Trillium) allow skipping fully empty causal blocks, cutting FLOPs by up to 50%.

- Sub-Tile Masking:

Pre-computing dense vs. sparse sub-tiles saves vector ALU operations in fused attention kernels.

MXU Dimension Optimization

- Systolic Hardware Fit:

TPU MXUs are 256x256 systolic arrays; head dim 128 yields only 25%–50% packed utilization.

- Dimension Tuning:

Picking head dimensions ≥ 256 (or optimized $S \times V$ transpositions) boosts hardware utilization to >67%.

Key Takeaway: Modern attention co-designs algorithmic compression (MLA/GQA) with TPU systolic array dimensions

MXU (Matmul unit) utilization

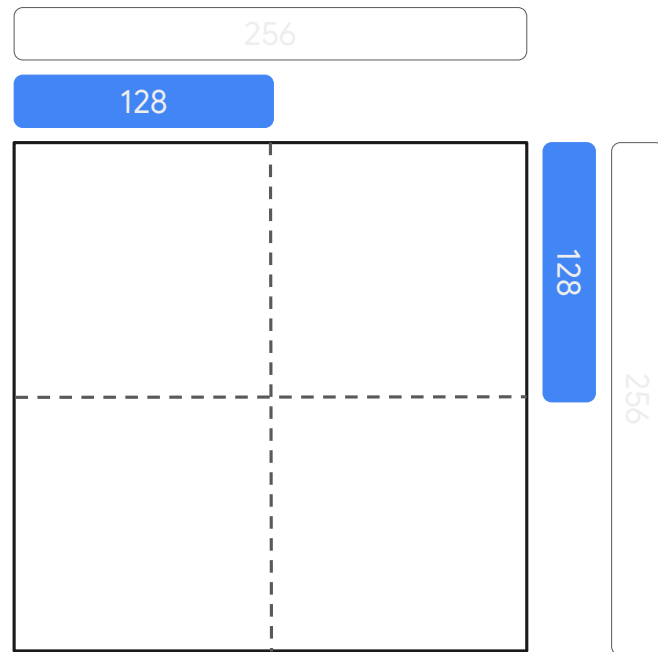
Matrix multiplication defined by (m, k, n)
 $(m, k) \times (k, n)$

In attention

- $Q K^T$ is $(s, h) \times (h, s) \rightarrow (m = s, k = h, n = s)$
- $S V$ is $(s, s) \times (s, h) \rightarrow (m = s, k = s, n = h)$

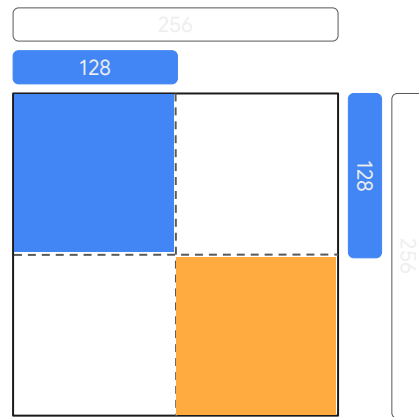
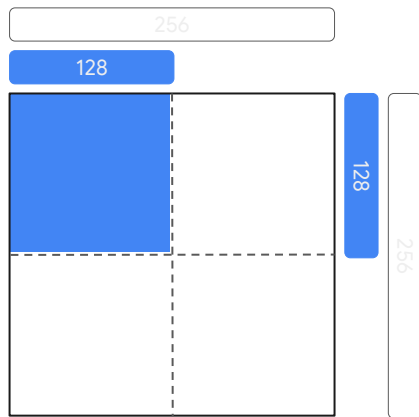
MXU has a physical size (on both GPU and TPU)

- GPU - more flexible, $m, k, n \sim 32-128$
- TPU - less flexible $m \geq 8, k = n = 256$ (recent gen)



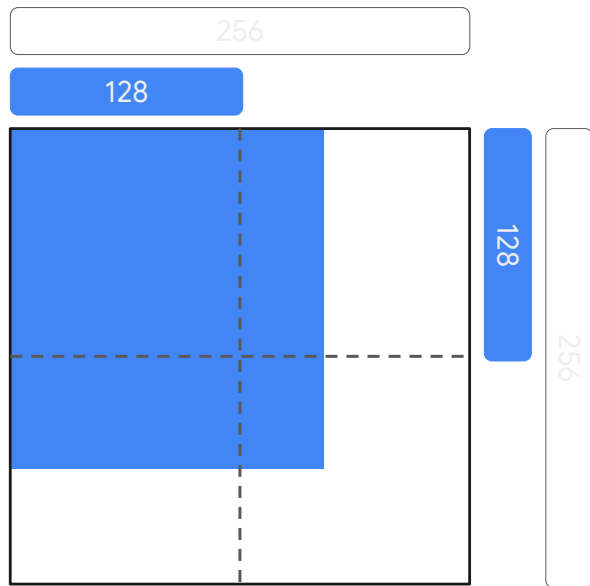
Attention with head dim = 128

- 128 is too small for the MXU
- Naively, only 25% of the MXU can be utilized
- Hardware supports packing - 50% utilization



Deepseek R1 - QK head dimension of 192

- Head dimension of 192 is better than 128
- But the area covered is around $(192 / 256)^2 = 56\%$
- Not much better than head dim = 128



S V product optimization for MXU

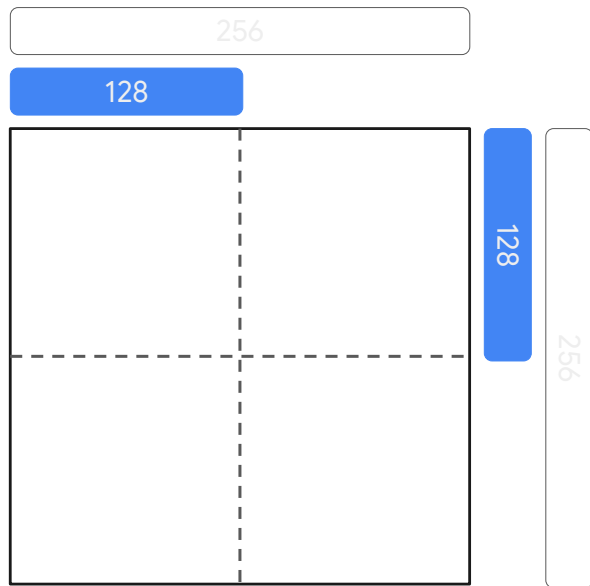
✓ Recall: $S V$ is $(s, s) \times (s, h) \rightarrow (m = s, k = s, n = h)$

✓ $k, n \geq 256$ for full performance, but $h = 128$

✓ Maybe compute $S V = (V^T S^T)^T$

✓ New product $(h, s) \times (s, s)$

✓ $m = h$, but $k = n = s$ (sequence length ≈ 4096)



A decorative graphic on the left side of the slide. It features a large blue circle with a white number '04' in the center. Above the blue circle is a green circle with a yellow star-like shape overlapping it. Below the blue circle are a yellow circle and a red circle, both partially visible.

04

Training Support

Pre-training

Supervised Fine Tuning (SFT)

Distillation

GRPO (RL)

The Spectrum of Training Objectives

1. Pretraining

- **Objective:** Next-token prediction:

$$L = -\sum \log P(x_t | x_{<t})$$

- **Scale:** Trillions of tokens.
- **Learns:** Syntax, world knowledge & algorithms.

2. SFT

- **Objective:** Instruction following on curated demos.

- **Loss Masking:**
Cross-entropy on target response tokens only.

- **Result:** Turns base model into a helpful assistant.

3. RL & Alignment

- **Algorithms:** DPO, PPO, and GRPO.

- **Coding Edge:** Unit tests provide verifiable ground-truth rewards!

- **Learns:** Multi-step reasoning & self-repair.

4. Multimodal

- **Modalities:** Vision (VLM), Video & Audio.

- **Architecture:** ViT encoder + trainable MLP projector.

- **Unified Space:** Interleaved image & code tokens.

MaxText provides unified, high-MFU recipes across every stage of the training lifecycle

Definition of post-training



Transforms a “raw” pre-trained model
(which predicts the next token) into
a helpful safe model.

Post-Training in MaxText: SFT, GRPO & GSPO

Supervised Fine-Tuning (SFT)

- **Instruction Tuning via Tunix:**

High-throughput training loop wrapping MaxText SPMD execution for NNX models.

- **Completion-Only Loss Masking:**

Cross-entropy loss applied strictly to response tokens (sft_train_on_completion_only=True), masking prompt context.

- **Zero-Waste Packing:**

Combines variable-length instruction/code examples into fixed sequences using Grain.

Reinforcement Learning: GRPO vs. GSPO

- **Group Relative Policy Optimization (GRPO):**

Token-level importance ratios & clipping over k sampled completions with group-normalized rewards.

- **Group Sequence Policy Optimization (GSPO):**

Sequence-level likelihood ratios weight all tokens equally; eliminates token noise and natively stabilizes MoEs without routing replay!

- **Unified Architecture:**

MaxText model trainer + vLLM inference sampler + Pathways CPU orchestration.

Key Takeaway: MaxText + Tunix provides end-to-end alignment: from completion-masked SFT to stable sequence-level GSPO

Overview of supervised fine-tuning (SFT)



Core concept

Adapting pre-trained models using **labeled instruction/response** pairs.



Objective

Instruction following: transforming raw predictors into helpful assistants.



Loss

Standard **Cross-Entropy Loss** applied to completion tokens only (user prompts are masked out).



Infrastructure

Combines **Tunix** (Training Loop) with **MaxText** (High-perf TPU execution).

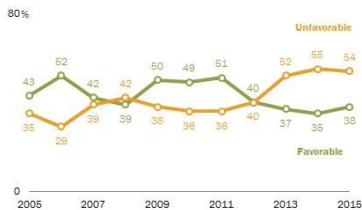
Multimodal Supervised Fine-Tuning (SFT)

Tokens: [User Prompt] [Vision Soft Tokens (256 - 8192)] [Target Response] [EOS]

Loss Mask: [0 0 0 0 0 0 0] [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0] [1 1 1 1 1 1 1 1 1 1] [1]

Objective: Cross-Entropy Loss evaluated exclusively on Target Response tokens

ChartQA

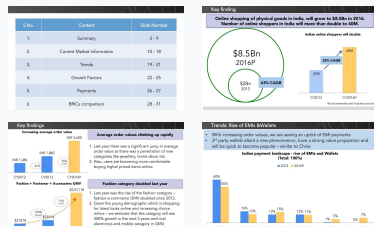


Q: Is the value of Favorable 38 in 2015?

A: Yes

SlideVQA

(Multi-image)



Q: How much in dollar is Online shopping in India in 2013?

A: 2Bn

coco-narratives

(Captioning)



Q: Describe this image

A: A bench with potted plants sitting on top of it

LLaVA-Video-178K

(Varied length & res)



Q: What can you see and hear?

A: A person dragging an Angry Birds suitcase

Knowledge distillation overview

A compression technique where a small "student" model learns to mimic the behavior of a large "teacher" model.



Efficiency: Cheaper-to-serve



Latency: Enables high-quality inference with lower latency



Performance: Use a a draft model with speculative decoding

Knowledge distillation primary methods

Offline (dataset generation)

- Teacher pre-generates a static dataset
- Student learns from it later via standard SFT

Online (logit matching)

- Student and Teacher run simultaneously
- Student minimizes kl divergence from live Teacher logits

Reinforcement learning overview

Concept: Shifts the training objective from "mimicking a reference" (SFT) to "maximizing a reward signal" (RL). The model learns what is good by interacting in the environment (trial and error).

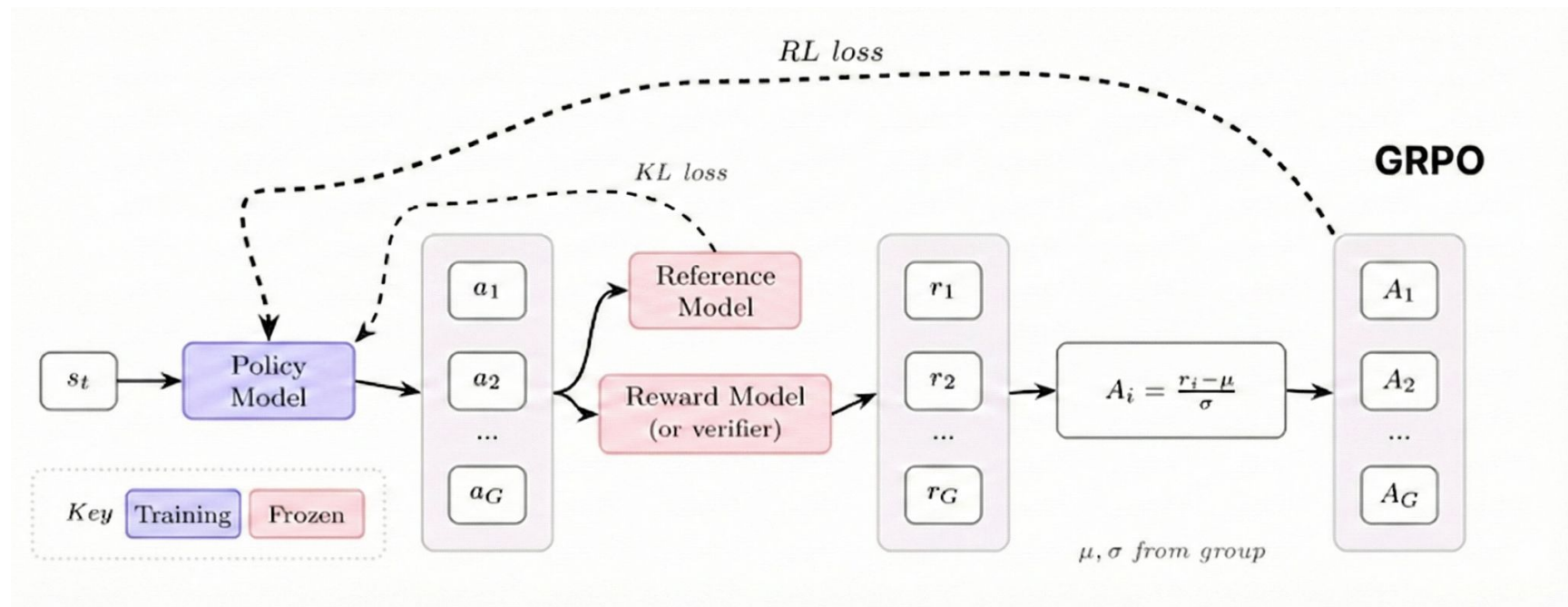
Why use RL for post-training?

- **Reasoning and correctness:** Critical for tasks like math or coding with a clear "right" answer, but many ways to derive it.
- **Alignment:** Enforcing safety guidelines or stylistic preferences where generating perfect human examples (SFT data) is difficult or expensive.

Key inputs

- **Prompts:** A dataset of queries (e.g., math questions) without target answers.
- **Reward function:** A mechanism to score the model's output (e.g., a deterministic rule checker or a separate reward model).

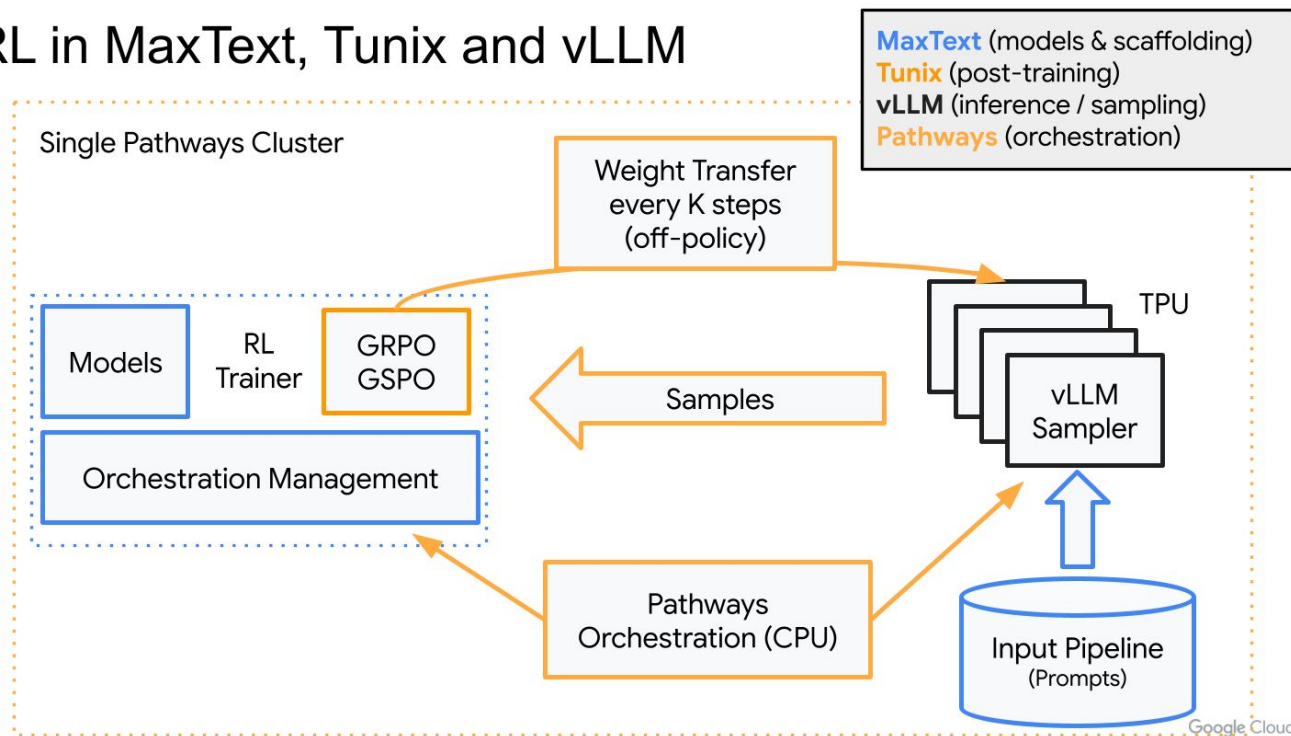
RL algorithms (GRPO)



Unified Post-Training Stack: MaxText, Tunix, vLLM & Pathways

Proprietary + Confidential

RL in MaxText, Tunix and vLLM





Colab!

Hands-On Quickstart: Running MaxText on Cloud TPUs

Step 1: Access TPU VM

- SSH into Cloud TPU VM:

```
gcloud compute tpus tpu-vm \  
  ssh <tpu-name> \  
  --zone=europe-west4-b
```

- Verify TPU Chips:

```
python3 -c "import jax; \  
  print(jax.devices())"
```

Step 2: Environment Setup

- Clone MaxText Repo:

```
git clone https://github.com/  
  google/maxtext.git  
cd maxtext
```

- Install Dependencies:

```
uv pip install maxtext[tpu]==0.2.4  
--resolution=lowest  
install_tpu_pre_train_extra_deps
```

Step 3: Launch Training

- Run Gemma / Qwen Demo:

```
python3 -m \  
  maxtext.trainers.train \  
  maxtext/configs/base.yml \  
  model_name=gemma3-4b \  
  run_name=ucla_demo \  
  steps=50 \  
  per_device_batch_size=2
```

Pro-Tip: Store temporary checkpoints and HuggingFace cache in /dev/shm for fast RAM disk I/O

JAX 101

[Colab](#)



MaxText for LLM Training/Inference

[Colab](#)



Final Remarks

01 Model Architecture

02 Performance Optimization

03 Improve Task Performance

04 Agentic Training



DeepSeek-V4

Toward Qwen

2026-04-22

Qwen3.5-Omni Technical Report

We present
Experts (M
DeepSeek-
one million
timization
and Heavy
Constraints
and the M
both mode
post-traini
Max, the n
open mode
highly effi



In this work,
model family
scales to hu
leveraging a
million hour
capabilities. Q
understandi
3.1 Pro in ke
Architecture
framework
model facili
standing an

Google DeepMind

Gemma 4 Technical Report

Gemma Team, Google DeepMind¹

We introduce Gemma 4, a new generation of open-weight, natively multimodal Gemma model family. Designed to advance compute efficiency and reasoning, features dense and Mixture-of-Experts architectures, ranging from 2.3B to 3 improved vision and audio encoders for all model sizes, we propose a unified for our 12B model, which ingests raw audio and image patches. Furthermore, mode, enabling Gemma models to generate reasoning traces prior to respond speed, memory, and compute efficiency, as well as long-context abilities through Gemma 4 establishes a leap in performance across STEM, multimodal, and and rivals larger, frontier open models in human-rated tasks.

1. Introduction

The rapid evolution of large language models

sharing ([Shazeer, 20](#)
as values in global lay
these optimizations r

Resources & Further Reading

MaxText Official Repository

- **GitHub:** github.com/google/maxtext
- **Includes:** Model zoo YAML configs, HF ↔ Orbx converters, SFT/RL recipes & benchmarks.

JAX & The Scaling Book

- **Scaling Book:** jax-ml.github.io/scaling-book
- **Includes:** Roofline math, TPU hardware architecture, SPMD sharding & transformer calc.

High-Performance LLMs Course

- **Course Materials:** github.com/rwitten/HighPerfLLMs2024
- **Includes:** Deep-dive lectures on systolic arrays, optical interconnects & MFU tuning.

Google Cloud TPU Documentation

- **Cloud Docs:** cloud.google.com/tpu/docs
- **Includes:** TPU v5e/v5p/v6e specs, Multislice XPK orchestration & TRC academic credits.

All frameworks, model recipes, and scaling guides covered today are open-source

Conclusion & Open Q&A

Core Lecture Takeaways

- **1. Co-Design Wins:** Peak efficiency requires co-designing silicon (TPU MXUs + OCS), compiler (XLA), and framework (JAX).
- **2. Transparent Scale with MaxText:** Pure Python/JAX delivers >60% MFU from a single 8-chip VM to 4,096+ chip pods.
- **3. Coding Intelligence is the Frontier:** Executable sandboxes provide verifiable rewards for self-improving RL agents.

Let's Connect & Discuss!

- **Speaker:** Hengtao Guo (@hengtaoguo)
- **Team:** Google Cloud TPU & MaxText Core Team
- **Open Source:** github.com/google/maxtext
- **Questions & Discussion:** Best of luck with your UCLA course projects!

Q&A

